

Handwritten Document Recognition

Architecture, Stitching Limitation & How LLMs Overcome It

Reference document — CNN + Transformer Decoder pipeline

1. How the CNN + Transformer Decoder Works

The architecture in the diagram is a full-page handwritten text recognition system. Unlike traditional OCR pipelines that segment text line-by-line, this model processes the entire page image in a single forward pass, using cross-attention to maintain full visual context throughout decoding.

1.1 Pipeline — step by step

#	Component	What it does
1	Raw image input	Full handwritten page scan — no pre-segmentation, no line cropping
2	ResNet CNN encoder	Processes entire image as 2D patches → spatial feature map ($H/32 \times W/32 \times 2048$)
3	Flatten + 2D position encoding	Each patch gets (row, col) coordinate — preserves spatial layout across the whole page
4	Stacked Transformer Decoder (×N)	Causal self-attention on generated tokens · Cross-attention to full image feature map · Add & Norm · FFN
5	Softmax over vocabulary	Generates tokens one by one, each attending to the whole image — no stitching required

1.2 Key design choices

- **2D position:** 2D positional encoding on image patches preserves spatial layout — the model knows which patch is top-left vs bottom-right, critical for reading order.
- **Cross-attention:** Cross-attention at every decoder layer means each generated token can attend to any part of the original image — a word at the bottom can reference a formula at the top.
- **Causal masking:** Causal (masked) self-attention on the output side ensures each predicted token only sees previously generated tokens, maintaining left-to-right generation order.
- **Teacher forcing:** Teacher forcing during training: ground-truth tokens are fed as decoder input, making training stable and fast. At inference, the model generates autoregressively.

2. The Stitching Limitation

The research literature (Figures 1 and 2 referenced in [3, 4]) describes a fundamental failure mode in older HTR systems: stitching error propagation. Understanding this is essential for appreciating why the CNN + Transformer architecture represents a significant advance.

2.1 What older systems did

Traditional HTR pipelines were built as a cascade of three independent models:

Step 1 — Line detection	Step 2 — OCR per line	Step 3 — Stitch lines
A segmentation model crops each row of text out of the full page as an independent image strip.	Each cropped line is fed to a separate OCR model in isolation. No surrounding context is available.	Transcriptions are concatenated in reading order. Errors from each stage compound at this point.

2.2 Why stitching fails on complex handwriting

The image provided illustrates exactly why stitching fails — it contains four semantically distinct regions:

- Panel (a): Graph theory diagram with symbolic notation and handwritten mathematical explanation.
- Panel (b): Python pseudocode with indentation-dependent semantics spanning multiple lines.
- Panel (c): Organic chemistry structural diagram with accompanying explanation of pi-bond rotation.
- Panel (d): Ksp chemistry calculation with exponent notation and a written conclusion.

When panel (b) is cropped line-by-line and each line is transcribed independently, the model has no way to know that the indented lines below `While count < N/2:` are the loop body. It cannot resolve abbreviations that are clarified by earlier lines. It cannot detect that `comp = predictions[k][1]` references a structure defined three lines above.

The stitch step then concatenates these isolated transcriptions, and errors from incorrect segmentation (a clipped ascender, a misread indent) cascade into every subsequent line. A single bad crop early in the document propagates all the way to the end.

2.3 How the CNN + Transformer architecture eliminates stitching

The diagram model removes stitching entirely by keeping the full 2D feature map alive through the entire decoding process:

- No line segmentation model is needed — the full page is processed as one image.

- The 2D positional encoding means the model knows the spatial relationship between every patch.
- Cross-attention at each decoder layer allows any generated token to reference any image patch, including patches from completely different regions of the page.
- There is no stitch step. The output sequence is generated in a single autoregressive pass with complete image context at every step.

3. Remaining Limitations of the Diagram Architecture

Despite eliminating the stitching problem, the CNN + Transformer Decoder still has four significant limitations that the research literature acknowledges:

Limitation	Detail
Resolution ceiling	ResNet downsamples by 32×. Small superscripts (like the -4 exponent in panel d) can be lost in the coarse feature map.
Multi-modal content	The image contains graph theory, pseudocode, chemical structures, and math. A single Softmax vocabulary head cannot faithfully represent all four domains simultaneously.
Long document degradation	Transformer cross-attention is $O(n^2)$ in patch count. A full A4 scan at 300 dpi produces thousands of patches — attention quality degrades and memory explodes.
No semantic understanding	The model transcribes characters. It has no idea that panel (b) is Python pseudocode or that panel (d) is a Ksp chemistry calculation — so it cannot use domain rules to self-correct.

4. How GPT-4o and Claude Address These Limitations

Modern large multimodal models (LMMs) like GPT-4o and Claude do not just transcribe — they understand. This distinction resolves all four remaining limitations listed above.

4.1 GPT-4o

GPT-4o	Native vision tokens with joint vision-language training GPT-4o encodes images as high-resolution visual tokens using a vision encoder trained jointly with the language model. Because vision and language share the same transformer weight space, GPT-4o can semantically interpret a structural chemical formula or indented pseudocode block — not just transcribe pixel shapes. It knows panel (b) is Python because it has seen millions of code images paired with
---------------	---

explanations during pretraining. The resolution limitation is addressed by tiling: GPT-4o splits high-resolution images into overlapping tiles, each processed at full detail, then merged by the model using its language context.

4.2 Claude

Claude

Multimodal context window with bidirectional vision-language attention

Claude treats images as first-class inputs inside the context window. Image tokens and text tokens attend to each other across the entire sequence bidirectionally. So if you add a question — 'what does the pseudocode return?' — the language reasoning about the answer and the visual tokens of the handwritten code inform each other simultaneously. This closes the semantic gap the diagram model cannot close with its unidirectional decoder. For the image shown, Claude can identify that panel (d) is computing molar solubility from a K_{sp} equilibrium expression and cross-check the arithmetic, while the diagram model would output only raw character sequences with no understanding of their domain meaning.

4.3 The fundamental difference

CNN + Transformer Decoder (diagram)	GPT-4o / Claude
Transcription machine	Reasoning machine that can also see
Converts pixel shapes to character sequences	Understands what the content means
Single vocabulary Softmax head	Domain-aware: knows code vs math vs chemistry
Unidirectional decoder cross-attention	Bidirectional vision-language attention
No self-correction using domain knowledge	Can apply domain rules to verify and correct output
Fails on mixed-modal documents	Handles mixed content as integrated semantic context

5. DeepSeek — Reading Words Clearly AND Understanding Diagrams

DeepSeek has released three vision models that are directly relevant to the handwritten document recognition problem. Uniquely among open-source models, DeepSeek-VL2 outperforms GPT-4o on OCR benchmarks (834 vs 736 on OCRBench), while DeepSeek-OCR introduces a novel vision-text compression approach that reduces token costs by 7-20x. Together they address both of the core requirements: reading handwritten text clearly, and understanding the meaning of diagrams.

5.1 DeepSeek-VL2 — Diagram Understanding

VL2	Mixture-of-Experts vision-language model with dynamic tiling DeepSeek-VL2 is a sparse MoE architecture based on DeepSeekMoE-27B with only 4.5B activated parameters per inference pass. Its key innovation for diagram understanding is the dynamic tiling vision encoding strategy. Rather than resizing the whole image to a fixed resolution (which crushes fine detail), VL2 splits the input into 384x384 tiles based on the image's actual aspect ratio, processes each tile through a shared SigLIP-SO400M-384 vision encoder, then stitches the tile features together with spatial position tokens before projecting into the language model via a two-layer MLP. This means a handwritten page with small superscripts in panel (d) or fine chemical bond lines in panel (c) is processed at full resolution — nothing is lost to downsampling. VL2 achieves 834 on OCRBench (vs GPT-4o's 736) and 93.3% on DocVQA.
-----	--

For diagram understanding specifically, VL2 supports visual grounding — it can not only read text but also locate and identify structural elements in the image using bounding box coordinates. This means for panel (a) it can identify the graph edges and node relationships, not just transcribe the handwritten explanation beside them.

5.2 DeepSeek-OCR — Clear Handwriting Transcription

OCR	Vision-text compression: 7-20x token reduction with high accuracy Released October 2025, DeepSeek-OCR takes a counterintuitive approach: it first converts document text into high-resolution image representations (DeepEncoder), then uses a compact MoE decoder (DeepSeek3B-MoE-A570M) to read those images back into structured text. This vision-text compression pipeline achieves 7-20x fewer tokens compared to feeding raw text, while maintaining high accuracy on complex handwriting. For your image, DeepSeek-OCR is invoked with domain-specific prompts: Convert the document to markdown for panel (b) pseudocode, OCR this image for handwritten text in panels (a) and (c), and Parse the figure for the chemical structure and mathematical notation in panels (c) and (d). It supports dynamic resolution up to $n \times 640 \times 640$ plus a 1024×1024 global context tile.
-----	---

5.3 Janus-Pro — Unified Understanding + Generation

Janus	Decoupled multimodal architecture for understanding and generation Janus-Pro separates the representation pathways for vision understanding and image generation — unlike models that force both through the same encoder. For understanding handwritten documents, it uses SigLIP-L at 384x384 as the vision encoder feeding into an autoregressive transformer. The decoupled design means the understanding pathway is not constrained by generation requirements, resulting in cleaner visual feature extraction. Janus-Pro is most useful when you need both to read the handwritten content AND generate a clean re-drawn version of a diagram — for example, converting the hand-drawn chemical structure in panel (c) into a clean SVG or LaTeX representation.
-------	--

5.4 How to integrate DeepSeek into your HTR pipeline

The recommended architecture combines all three DeepSeek models in a cascaded pipeline with role separation:

Stage	Model	Task	Handles
1 — Detect	DeepSeek-VL2	Visual grounding: locate text regions vs diagram regions with bounding boxes	Panel layout, spatial structure
2 — Transcribe	DeepSeek-OCR	Free OCR on handwritten text patches; Parse the figure on diagram patches	Panel (a)(b)(c)(d) text
3 — Understand	DeepSeek-VL2	VQA on diagram regions: what does this graph show, what is the chemistry	Semantic meaning of diagrams
4 — Reconstruct	Janus-Pro	Generate clean typed/drawn versions of hand-drawn diagrams	Panel (c) chemical structure

5.5 DeepSeek vs GPT-4o vs Claude — benchmark comparison

Benchmark	DeepSeek-VL2	DeepSeek-OCR	GPT-4o	Claude
OCRBench	834 ★ best	—	736	~780
DocVQA	93.3%	—	~92%	~91%
Token efficiency	Standard	7–20x reduction	Standard	Standard
Diagram grounding	Yes (bounding boxes)	Partial	Yes (implicit)	Yes (implicit)
Open source	Yes (MIT)	Yes (MIT)	No	No
GPU requirement	~40GB (small)	~24GB	API only	API only
Mixed-modal docs	Strong	Strong (OCR focus)	Strong	Strong

Note: DeepSeek-VL2 and DeepSeek-OCR are fully open-source (MIT license) and can be self-hosted on a single A100 40GB GPU, making them practical for on-premise AES deployments where data privacy or API costs are a concern.

6. Research Papers: Contributions & How to Use Them

Two papers by Sumeet S. Singh form the intellectual lineage of the architecture discussed throughout this document. Paper 1 (FPHR, ICDAR 2021) is the full-page HTR system. Paper 2 (Im2Latex / Teaching Machines to Code, 2018) is its predecessor — a single-formula LaTeX generator whose attention insights still apply directly today.

6.1 Paper 1 — Full Page HTR via Image to Sequence Extraction (ICDAR 2021)

What it contributes	How to use it in your AES system
Extractive problem formulation	Defines regions (text, math, drawing, deleted, table) and trains the model to extract only text while emitting markup tags for everything else. For AES: adapt this to label essay regions — <intro>, <thesis>, <body>, <evidence>, <weak-arg>, <conclusion>. Joint training on scores + region tags forces the shared trunk to learn essay structure, improving all three metric heads.
No image segmentation required	The ResNet encoder + 2D positional encoding processes the full page in one pass. Cross-attention at every decoder layer gives every generated token access to every image patch. Apply this to handwritten exam answer scoring: no pre-processing pipeline, no line detection, no stitching — the model handles curved lines, diagrams, and side-notes natively.
Auxiliary markup generation	The model emits <math>, <table>, <drawing>, <deleted-text> tags alongside transcribed text. For AES: emit <weak-arg>, <off-topic> tags as part of the scoring output. These tags become part of your XAI feedback without any separate classifier — the scoring and explanation are generated in one forward pass.
Local causal self-attention (window=50)	Limits the decoder self-attention span to 50 past positions, saving ~12% batch memory with no accuracy loss. Apply to your CNNTransformerScorer: since you only have one [SCORE] query token, self-attention is trivially bounded. For the TextCNN encoder processing long essays, a sliding window of 50 tokens saves significant GPU memory, enabling larger batch sizes during training on your HPC cluster.
Synthetic data augmentation	Generates unlimited synthetic essay images from text corpora using 34 fonts × multiple sizes (114 combinations). For AES: render your scoring rubric texts, student writing corpora, and WikiText into synthetic handwritten-style images during training. This dramatically reduces overfitting on your ~13K Free Form Answers dataset and builds a robust implicit language model across subject domains.
Character-level vocabulary	Uses character-level tokens specifically to handle subject-specific terminology, proper nouns, and domain symbols without out-of-vocabulary failures. For STEM essay scoring where students write equations, chemistry notation, and domain terms inline, switch from BPE/WordPiece to a char or char+word hybrid tokeniser. FPHR achieves 7.6% CER vs 14.4% for the best cloud API using this approach.

6.2 Paper 2 — Teaching Machines to Code: Neural Markup Generation (2018)

What it contributes	How to use it in your AES system
Attention = free bounding box (key insight)	The paper proves that cross-attention weights directly localise the symbol being transcribed — without bounding box training data. Alpha_t marks the exact image region corresponding to output token $y_{\hat{t}}$. Apply this to AES XAI: replace SHAP kernel approximations with direct cross-attention weight extraction. Each weight maps to a CNN encoder position → character offset. The result is exact, fast, and grounded — the model tells you which essay spans drove the score.
Init Model for grounded decoder start	An MLP reads the full mean-pooled encoder output and produces the initial decoder hidden state C_0 before generation begins. Without it, performance drops ~0.9% BLEU. For AES: instead of a random learnable score_query parameter, use an InitModel MLP that reads mean-pooled CNN encoder output and projects to d_{model} . This gives the [SCORE] query a semantically grounded starting point representing the essay's overall quality before cross-attention begins.
Deep output layer with skip connections	The output layer receives skip connections from both the LSTM input (attention context z_t) and previous output token embedding $E*y_{t-1}$. Adding skip connections yielded a 2% BLEU improvement. For AES: add skip connections from the CNN encoder mean-pool directly to your regression heads, bypassing the transformer decoder. This gives the scoring heads both a global essay view (from the encoder skip) and the refined semantic view (from the decoder output).
Sharp focal-region attention (I2L-NOPOOL)	The no-pooling variant produces much sharper, more localised attention focal-regions than the pooled variant. Sharper attention = more precise XAI span localisation. For AES: use the no-pooling CNN encoder variant when you need fine-grained span-level explanations (individual sentence attribution). Use the pooled variant when you only need paragraph-level feedback and want to save memory.
Masked mean pooling for Init Model	Mean-pooling with a mask to exclude padding positions produces a cleaner essay representation than naive mean-pooling. For variable-length essays (your dataset ranges from near-zero to 1100 chars), padding-aware mean pooling in the InitModel significantly reduces noise in the initial score_query vector — especially important for short essays where padding dominates the sequence.
ADAM hyperparameter tuning	The paper found that default ADAM $\beta_1=0.9$, $\beta_2=0.99$ caused choppy validation curves with frequent down-spikes. Reducing both moments to $\beta_1=0.5$, $\beta_2=0.9$ fixed instability. For AES training: if your validation QWK curves show sudden drops mid-training, reduce ADAM momentum — this is the documented fix from this paper, not a generic suggestion.

6.3 Priority implementation order for your AES system

Based on expected impact and implementation effort, here is the recommended order to apply these contributions:

#	Improvement	From paper	Expected impact
1	Attention-based XAI (replace SHAP)	Im2Latex (2)	High — exact span attribution, 10x faster than SHAP, interpretable by design
2	Init Model for	Im2Latex (2)	Medium — consistent +0.5–1% QWK, especially for

	score_query		short essays
3	Auxiliary markup tags for essay structure	FPHR (1)	High — joint training improves all three metric heads + free structural labels for feedback
4	Character-level / hybrid tokeniser	Both	High for STEM essays — eliminates OOV on equations, chemistry, domain terms
5	Synthetic data augmentation	FPHR (1)	High — essential if your labelled dataset is under 50K samples
6	Local self-attention window (w=50)	FPHR (1)	Medium — 12% memory saving, enables larger batches on same GPU
7	ADAM beta tuning (0.5 / 0.9)	Im2Latex (2)	Low-medium — prevents training instability, especially on small datasets
8	Skip connections to regression heads	Im2Latex (2)	Medium — +2% on output quality, one-line code change

7. Summary

The CNN + Transformer Decoder architecture represents a major advance over line-by-line OCR by eliminating the stitching problem through whole-page cross-attention. For simple, single-domain handwritten documents it performs well. However, for complex mixed-modal handwriting like the image analysed here — combining graph theory, pseudocode, chemistry, and mathematics — four limitations remain: resolution loss, multi-modal vocabulary constraints, quadratic attention cost at scale, and the absence of semantic understanding.

GPT-4o and Claude address all four limitations not by improving the transcription pipeline, but by fundamentally changing the task: instead of converting pixels to characters, they understand images with language. The stitching problem is solved as a side-effect of having a unified semantic model of the whole page — there is nothing to stitch because the document is never split.

DeepSeek adds a critical open-source dimension. DeepSeek-VL2 outperforms GPT-4o on OCR benchmarks while offering visual grounding for diagram element detection. DeepSeek-OCR delivers a 7-20x token compression breakthrough for document-heavy workloads. Used together in a 4-stage pipeline — detect, transcribe, understand, reconstruct — the three DeepSeek models can handle both clear word reading and full diagram comprehension, entirely on-premise, without API dependence.

The two source papers (FPHR 2021 and Im2Latex 2018) contribute six concrete, tested improvements to the AES scoring system: attention-based XAI span localisation, a grounded Init Model for the score query, auxiliary essay structure markup, character-level tokenisation for STEM robustness, synthetic data augmentation, and local self-attention for memory efficiency. All eight improvements are ranked by expected impact in Section 6.3, with corresponding implementation files available in the accompanying codebase.

